

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-ac13ada23fbe65ad1.jsonl`  
- SHA-256: `dc31eb3838a5c76a0b7054b065b796a737f90d2a8a95c7f1767d7695f58218e4`  
- Source modified: `2026-06-06T15:22:31+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d4359b2a-456`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:20:37.521Z)

You are mapping ONE subsystem of the "badminton-highlight-indexer" Python repo (cwd is the repo root) for a dense CODE MAP that a future AI engineer will read to ramp up FAST on the codebase without crawling files.

Subsystem: Stitcher, tools, utils, validators
Files: backend/stitcher/compiler.py, backend/tools/rally_slicer.py, backend/utils/video.py, backend/utils/geometry.py, backend/validators/base.py, backend/validators/blur.py, backend/validators/format.py, backend/validators/pts_continuity.py, backend/validators/relevance.py, backend/validators/resolution_fps.py, backend/validators/scene_cut.py

READ each listed file (use Read; use Grep/Glob to resolve any directory globs to actual files). Then return, per file:

- path (exact, repo-relative, forward slashes)
- role: one terse line ? what the file is for
- symbols: the IMPORTANT classes / functions / dataclasses / module-level constants & registries. For each: name, kind (class|func|dataclass|const|registry), a terse signature (params + return where useful), and a one-line role. SKIP trivial private helpers and boilerplate.
- contracts: exact data shapes this file defines or consumes (dict keys, CSV column orders, tuple layouts, dataclass fields, return types). Be precise ? these are the cross-module interfaces.
- edges: calls/imports to OTHER files that matter for dataflow, each as "thisSymbol -> otherFile.thing (why)".
- gotchas: non-obvious constraints, footguns, platform/WSL/GPU/threadpool specifics, dead paths, defaults that bite.

Rules: be terse and ACCURATE. Every symbol and path MUST actually exist in the file ? do NOT invent or guess. If something is notable but you're unsure, omit it. Return via the structured schema.

2. assistant (2026-06-06T15:20:40.384Z)

I'll map this subsystem. Let me read all the listed files in parallel.

3. user (2026-06-06T15:20:41.339Z)

```
1     import os  
2     import subprocess  
3     import tempfile  
4     from typing import List, Tuple, Union  
5  
6     class VideoCompiler:  
7         def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
```

```

0.5):
8         self.output_dir = output_dir
9         self.end_padding = end_padding
10        self.begin_padding = begin_padding
11        os.makedirs(output_dir, exist_ok=True)
12
13        @staticmethod
14        def _parse_time(val: Union[int, float, str]) -> float:
15            """Normalizes a timestamp value to float seconds.
16            Handles: float/int (pass-through), plain numeric strings ("12.4"),
17            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
18            This mirrors the parse_time logic used in gemini.py to ensure
19            the stitcher can always consume whatever format the indexer produced."""
20            if isinstance(val, (int, float)):
21                return float(val)
22            if isinstance(val, str):
23                val = val.strip()
24                if ":" in val:
25                    # Handle MM:SS, HH:MM:SS, etc.
26                    parts = val.split(":")
27                    parts.reverse()
28                    total = 0.0
29                    for i, p in enumerate(parts):
30                        try:
31                            total += float(p) * (60 ** i)
32                        except ValueError:
33                            pass
34                    return total
35                try:
36                    return float(val)
37                except ValueError:
38                    print(f"[COMPILER WARNING] Could not parse timestamp value '{val}' as a
39                    number. Defaulting to 0.0.")
40                    return 0.0
41                print(f"[COMPILER WARNING] Unexpected timestamp type {type(val).__name__} for
42                value '{val}'. Defaulting to 0.0.")
43                return 0.0
44
45        def _get_video_duration(self, video_path: str) -> float:
46            """Probes the video file to get its total duration in seconds.
47            Returns 0.0 if it cannot be determined."""
48            try:
49                result = subprocess.run(
50                    [
51                        "ffprobe", "-v", "error",
52                        "-show_entries", "format=duration",
53                        "-of", "default=noprint_wrappers=1:nokey=1",
54                        video_path
55                    ],
56                    capture_output=True, text=True, check=True
57                )
58                return float(result.stdout.strip())
59            except Exception as e:
60                print(f"[COMPILER WARNING] Could not determine video duration via ffprobe:
61                {e}")
62                return 0.0
63
64        def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
65        float]], output_filename: str) -> str:
66            """
67            Extracts selected segments from a raw video and stitches them together
68            into a seamless highlights file, applying end_padding to prevent abrupt endings.
69            """
70            if not segments:
71                raise ValueError("No highlight segments provided to compile.")

```

```

68
69         if not os.path.exists(raw_video_path):
70             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
71
72         output_path = os.path.join(self.output_dir, output_filename)
73
74         # Probe video duration so we can detect out-of-range segments
75         video_duration = self._get_video_duration(raw_video_path)
76         if video_duration > 0:
77             print(f"[COMPILER] Source video duration: {video_duration:.2f}s")
78         else:
79             print("[COMPILER WARNING] Video duration unknown. Cannot validate segment
boundaries against video length.")
80
81         # We will create temporary clips and stitch them using FFmpeg concat demuxer
82         temp_clips = []
83         skipped_segments = []
84
85         # Create a temp directory within output_dir
86         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
87             print(f"[COMPILER] Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
88             for idx, (raw_start, raw_end) in enumerate(segments):
89                 # Normalize timestamps to float seconds ? handles float, int,
90                 # plain numeric strings, and colon-separated formats like "MM:SS"
91                 start = self._parse_time(raw_start)
92                 end = self._parse_time(raw_end)
93                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
94
95                 # Validate the segment times
96                 if start < 0:
97                     print(f"[COMPILER WARNING] {segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
98                     start = 0.0
99
100                 if start >= end:
101                     print(f"[COMPILER ERROR] {segment_label}: start_time ({start:.2f}s)
>= end_time ({end:.2f}s). Skipping invalid segment.")
102                     skipped_segments.append((idx, start, end, "start >= end"))
103                     continue
104
105                 # Check if segment extends beyond video duration
106                 if video_duration > 0:
107                     if start >= video_duration:
108                         print(f"[COMPILER ERROR] {segment_label}: start_time
({start:.2f}s) is beyond the video duration ({video_duration:.2f}s). "
109                             f"This segment cannot be extracted ? the video ran out.
Skipping.")
110                         skipped_segments.append((idx, start, end, "start beyond video
end"))
111                         continue
112
113                     if end > video_duration:
114                         original_end = end
115                         end = video_duration
116                         print(f"[COMPILER WARNING] {segment_label}: end_time
({original_end:.2f}s) exceeds video duration ({video_duration:.2f}s). "
117                             f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")
118
119                     clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
120
121                 # Precise sub-second trim using fast re-encoding to preserve stream

```

```

headers
122         padded_start = max(0.0, start - self.begin_padding)
123         padded_end = end + self.end_padding
124
125         # Clamp padded_end to video duration if known
126         if video_duration > 0 and padded_end > video_duration:
127             padded_end = video_duration
128             print(f"[COMPILER INFO] {segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
129                   f"Clamping to {video_duration:.2f}s (end padding partially
applied).")
130
131         trim_duration = padded_end - padded_start
132         print(f"[COMPILER] Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
133
134         cmd = [
135             "ffmpeg", "-y",
136             "-ss", str(round(padded_start, 3)),
137             "-to", str(round(padded_end, 3)),
138             "-i", raw_video_path,
139             "-c:v", "libx264",
140             "-preset", "superfast",
141             "-crf", "22",
142             "-c:a", "aac",
143             "-b:a", "128k",
144             clip_path
145         ]
146
147         try:
148             result = subprocess.run(cmd, capture_output=True, check=True)
149             # Verify the output clip was actually created and has content
150             if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
151                 temp_clips.append(clip_path)
152             else:
153                 print(f"[COMPILER ERROR] {segment_label}: FFmpeg exited
successfully but output clip is missing or empty. Skipping.")
154                 skipped_segments.append((idx, start, end, "empty output clip"))
155             except subprocess.CalledProcessError as e:
156                 stderr_msg = e.stderr.decode(errors='replace').strip()
157                 print(f"[COMPILER ERROR] {segment_label}: FFmpeg trim failed.")
158                 print(f"  Command: {' '.join(cmd)}")
159                 print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
160                 skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
161                 continue
162
163         # Report summary of skipped segments
164         if skipped_segments:
165             print(f"\n[COMPILER SUMMARY] {len(skipped_segments)} out of
{len(segments)} segments were SKIPPED:")
166             for seg_idx, seg_start, seg_end, reason in skipped_segments:
167                 print(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
168             print()
169
170         if not temp_clips:
171             raise RuntimeError(
172                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
173                 f"All {len(segments)} segments were skipped. "
174                 f"Check the errors above for details. "
175                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
176             )

```

```

177
178         print(f"[COMPILER] Successfully trimmed {len(temp_clips)}/{len(segments)}
clips. Preparing to stitch...")
179
180         # Create a list file for FFmpeg concat demuxer
181         list_file_path = os.path.join(tmp_dir, "clips.txt")
182         with open(list_file_path, "w") as f:
183             for clip in temp_clips:
184                 # Escape single quotes in path
185                 safe_path = clip.replace("'", "'\\'")
186                 f.write(f"file '{safe_path}'\n")
187
188         print("[COMPILER] Stitching clips together...")
189         # Stitch them together without re-encoding (since they were encoded with
identical settings)
190         concat_cmd = [
191             "ffmpeg", "-y",
192             "-f", "concat",
193             "-safe", "0",
194             "-i", list_file_path,
195             "-c", "copy",
196             output_path
197         ]
198
199         try:
200             result = subprocess.run(concat_cmd, capture_output=True, check=True)
201
202             # Final verification
203             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
204                 raise RuntimeError(
205                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
206                     f"but output file is missing or empty: {output_path}"
207                 )
208
209             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
210             print(f"[SUCCESS] Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
211             return output_path
212         except subprocess.CalledProcessError as e:
213             stderr_msg = e.stderr.decode(errors='replace').strip()
214             raise RuntimeError(
215                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
216                 f"  Command: {' '.join(concat_cmd)}\n"
217                 f"  FFmpeg stderr: {stderr_msg[-500:]}"
218             )
219

```

4. user (2026-06-06T15:20:41.847Z)

```

1     """Slice individual rally clips out of a (long) annotated video.
2
3     Use cases:
4     - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5       boundaries against the labels.
6     - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7       supervision the segmenter is tuned/adapted on for that specific video.
8
9     Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10    end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11    <out_dir>/<video_stem>__rally_<n>.mp4
12
13    Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14    shows the same lead-in/out the model is trained and evaluated against. The window
15    math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16    `slice_rallies`.

```

```

17     """
18     from __future__ import annotations
19
20     import argparse
21     import csv
22     import os
23     import subprocess
24     from dataclasses import dataclass
25     from typing import Dict, List, Optional, Tuple
26
27     from backend.eval.annotations import parse_timestamp
28
29     # Match VideoCompiler defaults so sliced clips frame a rally the same way.
30     BEGIN_PADDING = 0.5
31     END_PADDING = 1.0
32
33
34     @dataclass
35     class ClipWindow:
36         rally: str          # kept as a string to allow inserted ids like "7.5"
37         start: float        # padded, clamped
38         end: float          # padded, clamped
39
40
41     def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
42         """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43         for key in (decimal_key, mmss_key):
44             val = (row.get(key) or "").strip()
45             if val:
46                 try:
47                     return parse_timestamp(val)
48                 except ValueError:
49                     continue
50         return None
51
52
53     def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54         """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56         Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57         end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58         """
59         labels: Dict[str, Tuple[float, float]] = {}
60         with open(csv_path, newline="", encoding="utf-8") as f:
61             reader = csv.DictReader(f)
62             reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63 [])]
64
65             for row in reader:
66                 row = {(k or "").strip().lower(): v for k, v in row.items()}
67                 rally = (row.get("rally_number") or "").strip()
68                 if not rally:
69                     continue
70                 start = _read_time(row, "start_time", "start_mmss")
71                 end = _read_time(row, "end_time", "end_mmss")
72                 if start is None or end is None or end <= start:
73                     continue # missing/degenerate (e.g. a not-yet-filled MISSED row)
74                 labels[rally] = (start, end)
75         return labels
76
77
78     def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
79                             rally_numbers: List[str],
80                             begin_pad: float = BEGIN_PADDING,
81                             end_pad: float = END_PADDING,
82                             video_duration: Optional[float] = None) -> List[ClipWindow]:

```

```

81     """Apply padding and clamp to the video, for the requested rallies.
82
83     rally_numbers may be ["all"] to take every labeled rally (in time order).
84     Unknown rally ids are skipped (caller can diff to warn).
85     """
86     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
87         wanted = sorted(labels, key=lambda r: labels[r][0])
88     else:
89         wanted = [r for r in rally_numbers if r in labels]
90
91     windows: List[ClipWindow] = []
92     for rally in wanted:
93         start, end = labels[rally]
94         ps = max(0.0, start - begin_pad)
95         pe = end + end_pad
96         if video_duration is not None and video_duration > 0:
97             if ps >= video_duration:
98                 continue # entirely past the end
99             pe = min(pe, video_duration)
100         if pe <= ps:
101             continue
102         windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103     return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(
109             ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111             capture_output=True, text=True, check=True,
112             ).stdout.strip()
113         return float(out)
114     except (subprocess.SubprocessError, OSError, ValueError):
115         return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []
136     for w in windows:
137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,
145         ]

```

```

146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151             continue
152         if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153             written.append(out_path)
154             print(f"[slicer] rally {w.rally} -> {out_path}
[{{w.start:.3f}}-{{w.end:.3f}}s]")
155         return written
156
157
158     def main() -> None:
159         p = argparse.ArgumentParser(description="Slice rally clips from an annotated
video.")
160         p.add_argument("--video", required=True, help="Source video file")
161         p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
162         p.add_argument("--rallies", required=True,
163                         help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
164         p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
165         p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
166         p.add_argument("--end-pad", type=float, default=END_PADDING)
167         args = p.parse_args()
168
169         rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
170         written = slice_rallies(args.video, args.labels, rally_numbers,
171                                args.out_dir, args.begin_pad, args.end_pad)
172         print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
173
174
175     if __name__ == "__main__":
176         main()
177

```

5. user (2026-06-06T15:20:42.317Z)

```

1     import subprocess
2     import os
3     import shutil
4     import logging
5
6     logger = logging.getLogger(__name__)
7
8     def get_video_duration(video_path: str) -> float:
9         """Returns the exact duration of a video file in seconds using ffprobe.
10         Returns 0.0 on failure."""
11         try:
12             duration_output = subprocess.check_output([
13                 "ffprobe", "-v", "error", "-show_entries", "format=duration",
14                 "-of", "default=noprint_wrappers=1:nokey=1", video_path
15             ]).decode().strip()
16             return float(duration_output)
17         except Exception as e:
18             logger.error(f"Could not detect video duration for {video_path}: {e}")
19             return 0.0
20
21     def extract_video_chunk(video_path: str, start_time: float, duration: float,
22                             output_path: str, reencode: bool = False) -> bool:
23         """Extracts a chunk from a video file.
24
25         Args:
26             reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                       If False, uses stream-copy (fast but cuts at nearest keyframe).
28                       Use True for short clips where keyframe drift matters.

```



```

29     """
30     cmd = [
31         "ffmpeg", "-y",
32         "-ss", str(start_time),
33         "-t", str(duration),
34         "-i", video_path,
35     ]
36     if reencode:
37         cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
38     else:
39         cmd += ["-c", "copy"]
40     cmd.append(output_path)
41     try:
42         subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
43         return True
44     except Exception as e:
45         logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
46         return False
47

```

6. user (2026-06-06T15:20:43.216Z)

```

1     import math
2     from typing import Tuple
3
4     def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5         """Calculate the center (x, y) of a bounding box.
6         bbox format: (x1, y1, x2, y2)
7         """
8         x1, y1, x2, y2 = bbox
9         return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11     def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12         """Calculate the Euclidean distance between two points (x, y)."""
13         return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15     def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
16         """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
17         """
18         c1 = get_center(bbox1)
19         c2 = get_center(bbox2)
20         dist = get_distance(c1, c2)
21         return dist * fps
22
23     def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
24                                fps: float, frame_width: float) -> float:
25         """Calculate velocity as a fraction of frame width per second.
26
27         This makes the threshold resolution-independent:
28         - 0.15 means the center moved 15% of the frame width in one second.
29         - Works identically on 720p, 1080p, and 4K footage.
30
31         Args:
32             bbox1: Previous bounding box (x1, y1, x2, y2).
33             bbox2: Current bounding box (x1, y1, x2, y2).
34             fps: Effective frames per second (after frame-skipping).
35             frame_width: Width of the video frame in pixels.
36         """
37         raw_velocity = get_velocity(bbox1, bbox2, fps)
38         if frame_width <= 0:

```

```

39         return raw_velocity # Fallback to raw pixels if width unknown
40     return raw_velocity / frame_width
41
42     def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
43         """Calculate the Intersection over Union (IoU) of two bounding boxes.
44         bbox format: (x1, y1, x2, y2)
45         """
46         xA = max(boxA[0], boxB[0])
47         yA = max(boxA[1], boxB[1])
48         xB = min(boxA[2], boxB[2])
49         yB = min(boxA[3], boxB[3])
50
51         interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
52         if interArea == 0:
53             return 0.0
54
55         boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
56         boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
57
58         iou = interArea / float(boxAArea + boxBArea - interArea)
59         return iou
60

```

7. user (2026-06-06T15:20:43.697Z)

```

1     from abc import ABC, abstractmethod
2     from pydantic import BaseModel
3     from typing import List, Dict, Any, Optional
4
5     class ValidationResult(BaseModel):
6         validator_name: str
7         passed: bool
8         message: str
9         details: Dict[str, Any] = {}
10
11     class VideoValidator(ABC):
12         @property
13         @abstractmethod
14         def name(self) -> str:
15             """Unique identifier for this validator."""
16             pass
17
18         @property
19         @abstractmethod
20         def description(self) -> str:
21             """Human-readable explanation of what this validator checks."""
22             pass
23
24         @abstractmethod
25         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
26             """
27             Executes validation.
28             - video_path: Local path to the MP4 file.
29             - config: Dict parsed from config.json.
30             - context: Dynamic context dict shared across validators (stores shared values
like ffmpeg metadata).
31             """
32             pass
33
34     class ValidationRegistry:
35         def __init__(self):
36             self._validators: List[VideoValidator] = []
37

```

```

38     def register(self, validator: VideoValidator):
39         """Registers a new validator."""
40         self._validators.append(validator)
41
42     def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
43         """Runs all registered validators in sequence, accumulating results."""
44         results = []
45         context = {} # Shared context dictionary (e.g. for caching ffmpeg outputs)
46         for val in self._validators:
47             try:
48                 res = val.validate(video_path, config, context)
49                 results.append(res)
50             except Exception as e:
51                 results.append(ValidationResult(
52                     validator_name=val.name,
53                     passed=False,
54                     message=f"Validator threw an unexpected error: {str(e)}"
55                 ))
56         return results
57
58 # Shared global registry
59 registry = ValidationRegistry()
60

```

8. user (2026-06-06T15:20:44.177Z)

```

1     import cv2
2     from backend.validators.base import VideoValidator, ValidationResult
3     from typing import Dict, Any
4
5     class BlurValidator(VideoValidator):
6         @property
7         def name(self) -> str:
8             return "blur_check"
9
10        @property
11        def description(self) -> str:
12            return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
13
14        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15            cap = cv2.VideoCapture(video_path)
16            if not cap.isOpened():
17                return ValidationResult(
18                    validator_name=self.name,
19                    passed=False,
20                    message="Failed to open video file for blur analysis."
21                )
22
23            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
24            if total_frames <= 0:
25                cap.release()
26                return ValidationResult(
27                    validator_name=self.name,
28                    passed=False,
29                    message="Invalid frame count."
30                )
31
32            # Sample 15 frames evenly distributed across the video
33            num_samples = 15
34            step = max(1, total_frames // num_samples)
35
36            variances = []

```

```

37
38     for i in range(num_samples):
39         frame_idx = i * step
40         cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
41         ret, frame = cap.read()
42         if not ret:
43             continue
44
45         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46         # Calculate the Laplacian variance (standard focus metric)
47         laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
48         variances.append(laplacian_var)
49
50     cap.release()
51
52     if not variances:
53         return ValidationResult(
54             validator_name=self.name,
55             passed=False,
56             message="Could not read frames to perform blur verification."
57         )
58
59     avg_variance = sum(variances) / len(variances)
60
61     details = {
62         "avg_laplacian_variance": round(avg_variance, 2),
63         "min_laplacian_variance": round(min(variances), 2),
64         "max_laplacian_variance": round(max(variances), 2),
65         "sampled_frames_count": len(variances)
66     }
67
68     val_cfg = config.get("validation", {})
69     min_blur_threshold = val_cfg.get("min_blur_threshold", 100.0)
70
71     if avg_variance < min_blur_threshold:
72         return ValidationResult(
73             validator_name=self.name,
74             passed=False,
75             message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
76             details=details
77         )
78
79     return ValidationResult(
80         validator_name=self.name,
81         passed=True,
82         message="Video sharpness meets focus requirements.",
83         details=details
84     )
85

```

9. user (2026-06-06T15:20:44.667Z)

```

1     import subprocess
2     import json
3     import os
4     from backend.validators.base import VideoValidator, ValidationResult
5     from typing import Dict, Any
6
7     class FormatValidator(VideoValidator):
8         @property
9         def name(self) -> str:
10             return "format_check"
11

```

```

12     @property
13     def description(self) -> str:
14         return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16     def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17         if not os.path.exists(video_path):
18             return ValidationResult(
19                 validator_name=self.name,
20                 passed=False,
21                 message=f"File not found: {video_path}"
22             )
23
24         cmd = [
25             "ffprobe",
26             "-v", "error",
27             "-select_streams", "v:0",
28             "-show_format",
29             "-show_streams",
30             "-of", "json",
31             video_path
32         ]
33
34         try:
35             result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36             info = json.loads(result.stdout)
37
38             if not info.get("streams") or not info.get("format"):
39                 return ValidationResult(
40                     validator_name=self.name,
41                     passed=False,
42                     message="Failed to retrieve valid video streams from the file."
43                 )
44
45             # Store the metadata in context for subsequent validators to utilize
46             context["ffprobe_meta"] = info
47
48             stream = info["streams"][0]
49             fmt = info["format"]
50
51             details = {
52                 "format_name": fmt.get("format_name"),
53                 "duration": float(fmt.get("duration", 0)),
54                 "size_bytes": int(fmt.get("size", 0)),
55                 "codec_name": stream.get("codec_name"),
56             }
57
58             # Check if container name includes mp4
59             if "mp4" not in fmt.get("format_name", "").lower():
60                 return ValidationResult(
61                     validator_name=self.name,
62                     passed=False,
63                     message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
64                     details=details
65                 )
66
67             return ValidationResult(
68                 validator_name=self.name,
69                 passed=True,
70                 message="Video format is valid MP4 container.",
71                 details=details
72             )
73

```

```

74         except subprocess.CalledProcessError as e:
75             return ValidationResult(
76                 validator_name=self.name,
77                 passed=False,
78                 message="Video stream is corrupted or unreadable by ffprobe.",
79                 details={"error": e.stderr}
80             )
81         except Exception as e:
82             return ValidationResult(
83                 validator_name=self.name,
84                 passed=False,
85                 message=f"Unexpected error running ffprobe: {str(e)}"
86             )
87

```

10. user (2026-06-06T15:20:45.560Z)

```

1     import subprocess
2     from backend.validators.base import VideoValidator, ValidationResult
3     from typing import Dict, Any
4
5     class PTSContinuityValidator(VideoValidator):
6         @property
7         def name(self) -> str:
8             return "pts_continuity_check"
9
10        @property
11        def description(self) -> str:
12            return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
13            timeline gaps, jumps, or editing splices."
14
15        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
16        -> ValidationResult:
17            # Command to extract packet PTS and flags (extremely fast, works on H.264 &
18            H.265)
19            cmd = [
20                "ffprobe",
21                "-v", "error",
22                "-select_streams", "v:0",
23                "-show_entries", "packet=pts_time,flags",
24                "-of", "csv=p=0",
25                video_path
26            ]
27
28            try:
29                result = subprocess.run(cmd, capture_output=True, text=True, check=True)
30                output = result.stdout.strip()
31
32                if not output:
33                    return ValidationResult(
34                        validator_name=self.name,
35                        passed=False,
36                        message="No packet timestamps could be extracted from the video
37                        stream."
38                    )
39
40                timestamps = []
41                for line in output.split("\n"):
42                    line = line.strip()
43                    if line:
44                        parts = line.split(",")
45                        if len(parts) == 2:
46                            pts_str, flags = parts
47                            if "K" in flags: # Check if keyframe
48                                try:

```

```

45             timestamps.append(float(pts_str))
46         except ValueError:
47             continue
48
49     if len(timestamps) < 2:
50         return ValidationResult(
51             validator_name=self.name,
52             passed=True,
53             message="Video is too short to perform a PTS continuity check.",
54             details={"keyframes_count": len(timestamps)}
55         )
56
57     # Analyze gaps
58     max_gap = 0.0
59     backward_jumps = 0
60     gaps = []
61
62     for i in range(1, len(timestamps)):
63         gap = timestamps[i] - timestamps[i-1]
64         gaps.append(gap)
65         if gap < 0:
66             backward_jumps += 1
67         if gap > max_gap:
68             max_gap = gap
69
70     # An continuous GoPro recording usually has keyframe intervals (GOP size) of
71     # 1.0 to 4.0 seconds.
72     # If there's a keyframe gap larger than 8.0 seconds or a negative gap
73     # (backward jump), it's highly likely spliced.
74     details = {
75         "keyframes_count": len(timestamps),
76         "max_keyframe_gap_seconds": round(max_gap, 2),
77         "backward_jumps_detected": backward_jumps,
78         "first_keyframe": timestamps[0],
79         "last_keyframe": timestamps[-1]
80     }
81
82     if backward_jumps > 0:
83         return ValidationResult(
84             validator_name=self.name,
85             passed=False,
86             message="Video integrity compromise: Backward timeline jumps
87             (splicing) detected.",
88             details=details
89         )
90
91     # Threshold for sudden gap (e.g. 10.0 seconds)
92     if max_gap > 10.0:
93         return ValidationResult(
94             validator_name=self.name,
95             passed=False,
96             message=f"Video integrity compromise: Detected a timestamp gap of
97             {round(max_gap, 2)}s, indicating edited or missing content.",
98             details=details
99         )
100
101     return ValidationResult(
102         validator_name=self.name,
103         passed=True,
104         message="PTS timeline is continuous with no abnormal gaps or backward
105         jumps.",
106         details=details
107     )
108
109 except subprocess.CalledProcessError as e:

```

```

105         return ValidationResult(
106             validator_name=self.name,
107             passed=False,
108             message="Failed to execute ffmpeg PTS analysis.",
109             details={"error": e.stderr}
110         )
111     except Exception as e:
112         return ValidationResult(
113             validator_name=self.name,
114             passed=False,
115             message=f"Unexpected error in PTS validation: {str(e)}"
116         )
117

```

11. user (2026-06-06T15:20:46.059Z)

```

1     import cv2
2     import numpy as np
3     from backend.validators.base import VideoValidator, ValidationResult
4     from typing import Dict, Any
5
6     class RelevanceValidator(VideoValidator):
7         @property
8         def name(self) -> str:
9             return "relevance_check"
10
11        @property
12        def description(self) -> str:
13            return "Checks if the video content is relevant to badminton by scanning for
14            court color palettes and geometric features."
15
16        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
17        -> ValidationResult:
18            cap = cv2.VideoCapture(video_path)
19            if not cap.isOpened():
20                return ValidationResult(
21                    validator_name=self.name,
22                    passed=False,
23                    message="Failed to open video file for relevance validation."
24                )
25
26            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
27            if total_frames <= 0:
28                cap.release()
29                return ValidationResult(
30                    validator_name=self.name,
31                    passed=False,
32                    message="Invalid frame count."
33                )
34
35            # Sample 10 frames across the video
36            num_samples = 10
37            step = max(1, total_frames // num_samples)
38
39            green_ratios = []
40
41            # Get sport profile config
42            sport_name = config.get("sport", "badminton")
43            from backend.sports import sport_registry
44            try:
45                sport_profile = sport_registry.get(sport_name)
46                sport_rel_cfg = sport_profile.get_relevance_config()
47            except KeyError:
48                sport_rel_cfg = {}

```



```

48         val_cfg = config.get("validation", {})
49         rel_cfg = val_cfg.get("relevance", {})
50
51         # Read bounds from sport profile with fallback to user config/defaults
52         hsv_lower = np.array(rel_cfg.get("court_green_lower",
sport_rel_cfg.get("court_green_lower", [35, 40, 40])))
53         hsv_upper = np.array(rel_cfg.get("court_green_upper",
sport_rel_cfg.get("court_green_upper", [85, 255, 255])))
54         min_court_ratio = rel_cfg.get("court_pixels_min_ratio",
sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
55
56         for i in range(num_samples):
57             frame_idx = i * step
58             cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
59             ret, frame = cap.read()
60             if not ret:
61                 continue
62
63             # Downscale frame for speed
64             h, w = frame.shape[:2]
65             small_frame = cv2.resize(frame, (320, 240))
66
67             # Convert to HSV color space
68             hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)
69
70             # Mask court green pixels
71             mask = cv2.inRange(hsv, hsv_lower, hsv_upper)
72             green_pixels = cv2.countNonZero(mask)
73             total_pixels = mask.shape[0] * mask.shape[1]
74
75             ratio = green_pixels / total_pixels
76             green_ratios.append(ratio)
77
78         cap.release()
79
80         if not green_ratios:
81             return ValidationResult(
82                 validator_name=self.name,
83                 passed=False,
84                 message="Failed to extract frames for relevance checking."
85             )
86
87         avg_green_ratio = sum(green_ratios) / len(green_ratios)
88
89         details = {
90             "avg_green_court_ratio": round(avg_green_ratio, 4),
91             "max_green_court_ratio": round(max(green_ratios), 4),
92             "min_green_court_ratio": round(min(green_ratios), 4),
93             "green_threshold": min_court_ratio
94         }
95
96         # If a certain amount of court color is visible, consider it relevant
97         if avg_green_ratio < min_court_ratio:
98             return ValidationResult(
99                 validator_name=self.name,
100                 passed=False,
101                 message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
102                 details=details
103             )
104
105         return ValidationResult(
106             validator_name=self.name,
107             passed=True,

```

```

108         message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
109         details=details
110     )
111

```

12. user (2026-06-06T15:20:46.527Z)

```

1     from backend.validators.base import VideoValidator, ValidationResult
2     from typing import Dict, Any
3
4     class ResolutionFPSValidator(VideoValidator):
5         @property
6         def name(self) -> str:
7             return "resolution_fps_check"
8
9         @property
10        def description(self) -> str:
11            return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14            meta = context.get("ffprobe_meta")
15            if not meta or not meta.get("streams"):
16                return ValidationResult(
17                    validator_name=self.name,
18                    passed=False,
19                    message="Video metadata is missing. Run format_check validator first."
20                )
21
22            stream = meta["streams"][0]
23            width = int(stream.get("width", 0))
24            height = int(stream.get("height", 0))
25
26            # Parse frame rate (e.g. "30/1" or "30000/1001")
27            fps_str = stream.get("r_frame_rate", "0/1")
28            try:
29                num, den = map(int, fps_str.split("/"))
30                fps = num / den if den != 0 else 0
31            except ValueError:
32                fps = 0.0
33
34            details = {
35                "width": width,
36                "height": height,
37                "fps": round(fps, 2)
38            }
39
40            # Check bounds from config
41            val_cfg = config.get("validation", {})
42            min_w = val_cfg.get("min_resolution_width", 1280)
43            min_h = val_cfg.get("min_resolution_height", 720)
44            min_fps = val_cfg.get("min_fps", 29.9)
45
46            if width < min_w or height < min_h:
47                return ValidationResult(
48                    validator_name=self.name,
49                    passed=False,
50                    message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51                    details=details
52                )
53
54            if fps < min_fps:

```

```

55         return ValidationResult(
56             validator_name=self.name,
57             passed=False,
58             message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
59             details=details
60         )
61
62     return ValidationResult(
63         validator_name=self.name,
64         passed=True,
65         message="Video resolution and frame rate meet minimum standards.",
66         details=details
67     )
68

```

13. user (2026-06-06T15:20:47.010Z)

```

1     import cv2
2     import numpy as np
3     from backend.validators.base import VideoValidator, ValidationResult
4     from typing import Dict, Any
5
6     class SceneCutDensityValidator(VideoValidator):
7         @property
8         def name(self) -> str:
9             return "scene_cut_check"
10
11        @property
12        def description(self) -> str:
13            return "Analyzes visual frames to detect abrupt scene cuts, which indicate an
edited or compiled video."
14
15        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16            cap = cv2.VideoCapture(video_path)
17            if not cap.isOpened():
18                return ValidationResult(
19                    validator_name=self.name,
20                    passed=False,
21                    message="Failed to open video file using OpenCV."
22                )
23
24            fps = cap.get(cv2.CAP_PROP_FPS)
25            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
26            duration = total_frames / fps if fps > 0 else 0
27
28            if duration <= 0:
29                cap.release()
30                return ValidationResult(
31                    validator_name=self.name,
32                    passed=False,
33                    message="Invalid video duration."
34                )
35
36            # Sample at 2 frames per second to keep it fast
37            sample_interval = max(1, int(fps / 2))
38
39            cuts_detected = 0
40            prev_hist = None
41
42            frame_idx = 0
43            sampled_count = 0
44
45            # Adjust histogram comparison thresholds

```

```

46         # Using correlation: 1.0 is perfect match, < 0.6 indicates a sudden massive
scene change
47         cut_threshold = 0.6
48
49         while True:
50             cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
51             ret, frame = cap.read()
52             if not ret:
53                 break
54
55             # Calculate color histogram
56             hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
57             hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
58             cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)
59
60             if prev_hist is not None:
61                 # Compare correlation
62                 score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)
63                 if score < cut_threshold:
64                     cuts_detected += 1
65
66             prev_hist = hist
67             sampled_count += 1
68             frame_idx += sample_interval
69
70             # Cap check at 100 frames max (e.g. first 50 seconds or evenly spread) to
keep validation instant
71             if frame_idx >= total_frames or sampled_count > 100:
72                 break
73
74             cap.release()
75
76             # Compute cuts per minute
77             sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
78             cuts_per_minute = (cuts_detected / sampled_duration) * 60 if sampled_duration >
0 else 0
79
80             details = {
81                 "cuts_detected": cuts_detected,
82                 "sampled_frames": sampled_count,
83                 "cuts_per_minute": round(cuts_per_minute, 3)
84             }
85
86             val_cfg = config.get("validation", {})
87             max_cuts_rate = val_cfg.get("max_scene_cuts_per_minute", 0.5)
88
89             if cuts_per_minute > max_cuts_rate:
90                 return ValidationResult(
91                     validator_name=self.name,
92                     passed=False,
93                     message=f"Video splicing compromise: Detected {cuts_detected} cuts
({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of
{max_cuts_rate} cuts/min.",
94                     details=details
95                 )
96
97             return ValidationResult(
98                 validator_name=self.name,
99                 passed=True,
100                 message="Scene continuity check passed. No abnormal visual cuts detected.",
101                 details=details
102             )
103

```

All 11 files are read. Let me check the relevance validator's dependency on `backend.sports` to confirm the `get\_relevance\_config` contract, since it's a cross-module edge.

15. user (2026-06-06T15:20:52.797Z)

```
backend\sports\base.py:3:class SportProfile(ABC):
backend\sports\base.py:16:     def get_relevance_config(self) -> dict:
backend\sports\badminton.py:3:class BadmintonSportProfile(SportProfile):
backend\sports\badminton.py:65:     def get_relevance_config(self) -> dict:
backend\sports\__init__.py:15:     def get(self, name: str) -> SportProfile:
backend\sports\__init__.py:23:sport_registry = SportRegistry()
```

16. assistant (2026-06-06T15:20:54.852Z)

I have everything I need. Let me return the structured map.

17. user (2026-06-06T15:22:29.371Z)

Structured output provided successfully

18. assistant (2026-06-06T15:22:31.749Z)

Mapped all 11 files in the Stitcher/tools/utils/validators subsystem and returned the dense code map via StructuredOutput.